

Olist E-Commerce Intelligence Pipeline: PySpark-Enabled Multi-Dimensional Analytics

1. Student Details

Name	Student ID	Email ID
Haritha Ramadass	A00052071	Haritha.ramadass2@mail.dcu.ie

2. Link for the Dataset

Brazilian E-Commerce Public Dataset by Olist <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce/data>

3. Introduction and Motivation for the Pipeline and the Choice of Technologies

Customers find online purchasing simple: click 'buy,' pay, and wait for delivery. The Olist dataset's multiple files of orders, customers, products, sellers, payments, reviews, order components, and geolocations illustrate this complexity, making the data valuable yet challenging. This realistic fragmentation requires careful preprocessing to extract meaningful insights, which keeps readers focused on the project's technical depth.

The primary intent of our work was to produce a reliable end-to-end data pipeline that manages these dispersed sources by using PySpark to load each raw CSV file, perform thorough cleaning and null-value handling, and combine them into an enriched master dataset. This structured approach aims to instill confidence in peers about the pipeline's robustness, supporting in-depth Exploratory Data Analysis (EDA) to identify patterns in order timings, customer segmentation, payment behavior, product performance, geospatial trends, and review sentiment. For instance, we sought to address the following important practical questions: How long does it take for an order to be delivered? Which goods and payment options influence sales? What impact do delivery and location delays have on client satisfaction?

Our methodology reflects industry best practices for scalable data engineering. First, we created reusable PySpark functions and support tools for consistent type enforcement, filling in missing values with medians or averages, and data profiling (missing values and duplicates). Incorporating geolocation data collected at the ZIP code level and performing numerous joins to enrich orders with customer, seller, product, payment, and review details demonstrates the pipeline's scalability.

Hundreds of thousands of entries can be processed effectively without memory bottlenecks using PySpark's distributed DataFrame API, which should reassure professionals about handling large datasets efficiently.

The main technologies selected are still available for coursework while closely matching real-world cloud data engineering environments:

- **Visual Studio Code:** Our development environment was Visual Studio Code, which enabled simple, modular scripting and testing of pipeline stages, as well as straightforward result annotation and export.
- **Python:** For transformations and utility functions, Python offers comprehensible logic, and it is tightly integrated with PySpark for huge data processing.
- **PySpark:** Scalable CSV loading, null-value imputation, sophisticated joins, and feature engineering with distributed resources were all made possible, which were crucial for managing the large, multi-file dataset.
- **Matplotlib and Seaborn:** High-quality visualisations from processed Spark data were generated, displaying distributions, trends, and cluster insights that illuminate the underlying narrative.

With everything considered, this pipeline transforms unprocessed, fragmented data into a cleaned, cohesive dataset that supports scalable, reliable analytics. The use of Python and PySpark, which provide reproducibility and extensibility in a Visual Studio Code format that is perfect for experimentation and presentation, reflects a cloud-ready architecture typical in industry.

4. Related Work:

i. Order Timeline Analysis

Existing work, such as Philip Clemente's "Olist eCommerce Analysis and Dashboard" on Kaggle, analyses delivery times and delays through exploratory visualisations.

Link: <https://www.kaggle.com/code/philipclemente/olist-ecommerce-analysis-and-dashboard>

Our project differs by using PySpark to calculate detailed order stage intervals-purchase-to-approval, approval-to-shipment, shipment-to-delivery-using efficient timestamp arithmetic, and handling null timestamps for real-world scenarios like cancellations. This granularity enhances bottleneck detection, highlighting our innovative approach to practical challenges.

ii. Customer Analytics

K-means and DBSCAN clustering are applied to Olist customer purchasing behavior in Faroja's Customer Segmentation project on GitHub.

Link: <https://github.com/Faroja/Olist-Customers-Segmentation>

How our project differs: To create deeper, more multifaceted consumer categories that better reflect the variety in the real world, we combined demographic and geographic (zip code, city, state) characteristics with purchase frequency and spending data.

iii. Review Quality and Sentiment

By using ML models, studies like Vilem Knap's sentiment analysis connect product reviews to delivery and product qualities.

Link: <https://vilemknap.cz/applying-sentiment-analysis-for-product-reviews-and-predictions/>

How our project differs: To analyse how delivery and product category directly affect customer happiness, we integrated review scores, sentiment derived from textual complaint topics, and order fulfilment timelines into a single Spark dataset.

iv. Product and Seller Performance

Olist products and sellers are ranked by sales volumes and ratings in Tolamoye's GitHub project.

Link: <https://github.com/tolamoye/Olist-E-commerce-Data-Analysis>

How our project differs: To provide practical insights into the best- and worst-performing segments, we also examined product return rates and integrated indicators such as price, freight values, and category dimensions.

v. Geospatial Sales Mapping

Using regional data, business intelligence dashboards have displayed Olist sales and popular product categories by location.

Link: <https://dataviz.etublogs.usj.edu.lb/2020/12/02/business-intelligence-dashboards-olist-the-brazilian-e-commerce-website/>

How our project differs: For a more thorough regional performance study, we combined customer and seller geography data, showing not just sales volumes but also shipping delays and review scores across Brazilian states and cities.

Vi. Payment and Pricing Trends

The effects of different payment methods on order values and consumer behavior are investigated in exploratory projects.

Link:https://github.com/Analyst-Joan/Olist_Store_Exploratory_Data_Analysis/blob/main/README.md

How our project differs: We investigated the effects of pricing and freight costs on order success and satisfaction, as well as on instalment payment habits, integrating these into the Spark pipeline for integrated analysis.

5. Description of the Dataset

a. Source of the Data

The Olist Brazilian E-commerce dataset, a well-known public dataset widely available on platforms such as Kaggle, was used in this study. It contains real business data from approximately 100,000 customer orders placed on Olist, a central Brazilian online marketplace, in 2016 and 2018. The dataset provides a holistic view of the e-commerce process, with extensive transactional data on purchases, client profiles, product descriptions, payment methods, seller details, reviews, and geographic data linking zip codes to latitude and longitude coordinates. The data, which reflects the complexities and realities of an e-commerce company operating across multiple markets in Brazil, offers an opportunity for a thorough analysis of sales performance, consumer behaviour, and logistics.

b. Process of Extraction and Collection

Olist released the raw data as several CSV files, which were disseminated independently by the data domain for academic study and educational purposes. Olist gathered and compiled these files to depict the full customer buy lifecycle, including order creation, product delivery, and post-purchase comments. Each of the eight CSV files was loaded directly into a PySpark environment to extract the data for this project. This required using the `spark.read.csv` method in PySpark, which offers options to enable automatic schema inference and include column headers. By identifying and allocating appropriate information types, especially for numerical components such as costs, transaction amounts, and freight charges, this process reduced manual preparation errors. With PySpark's distributed infrastructure, these enormous datasets were successfully ingested, enabling scalable data processing. To create a unified, enriched dataset suitable for in-depth exploratory research, this preliminary extraction served as the basis for subsequent data cleaning, merging, and analysis.

6. Description of Data Processing

We carefully cleaned and profiled each CSV file individually before joining the datasets, ensuring reliable, precise joins in the future. This phase was essential, as incomplete or deceptive analytics could result from missing numbers, incorrect data types, or duplicate data.

1. Environment Setup

Utilizing PySpark for scalable distributed data processing, the project was created in Visual Studio Code. To handle cluster resources and SQL procedures for data manipulation, we first installed PySpark and set up a Spark session. This configuration enabled effective operations such as profiling, filtering, and transformations, and allowed the import of a large number of CSV files into Spark DataFrames.

```
# Imports Spark modules and Python libraries

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

import matplotlib.pyplot as plt
import seaborn as sns

import re
from collections import Counter

from IPython.display import display
from IPython.display import FileLink
from pyspark.sql import SparkSession
from pyspark.sql.functions import floor, col, to_timestamp, when, avg, stddev, hour, dayofweek, datediff, upper, sum, lit, array, explode, struct
from pyspark.sql.functions import to_date, month, year, count, mean, isnan, unix_timestamp, round

spark = SparkSession.builder \
    .appName("OlistCloudProject") \
    .getOrCreate()

print("PySpark version:", spark.version)
```

2) Loading Datasets

Using PySpark's spark, we loaded each of the eight datasets (orders, reviews, payments, items, customers, products, sellers, and geolocation). make use of the read.csv() method with header=True and inferSchema=True as options. This version reduced the risk of manual schema specification errors by ensuring automatic detection of numerical columns such as pricing, payment amounts, and freight charges.

```
# ----- 1. LOAD DATA -----
orders_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_orders_dataset.csv', header=True, inferSchema=True)
reviews_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_order_reviews_dataset.csv', header=True, inferSchema=True)
payments_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_order_payments_dataset.csv', header=True, inferSchema=True)
items_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_order_items_dataset.csv', header=True, inferSchema=True)
customers_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_customers_dataset.csv', header=True, inferSchema=True)
products_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_products_dataset.csv', header=True, inferSchema=True)
sellers_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_sellers_dataset.csv', header=True, inferSchema=True)
geo_df = spark.read.csv('/Users/shiva/Documents/PROJECTS/Cloud Computing Project /archive-5/olist_geolocation_dataset.csv', header=True, inferSchema=True)
```

3) Initial Data Profiling

We carried out thorough initial profiling for each dataset, which included:

- Previewing the first few rows using. Show (5) to examine data types and contents.

- To verify the existence and kinds of columns, particularly those utilized as keys in joins, schemas can be printed using the `printSchema()` function.
- Using the reusable utility function `show_null_counts(df)`, one may check each column for missing values.
- To comprehend distributions, min/max values, and anomalies in numerical fields, use `describe()` to run descriptive statistics.
- By aggregating all columns and filtering to find rows that appear more than once, duplicates can be found.

```
# Display the first 5 rows of the DataFrame to preview data
orders_df.show(5, truncate=False)

# Print all column names
print("Columns:", orders_df.columns)

# Print the schema (column names and data types)
orders_df.printSchema()

# Print count of missing values (nulls) for each column using your helper function
print("NULL counts per column (list):")
show_null_counts(orders_df)

# Print descriptive statistics
orders_df.describe().show(truncate=False)

# Find and count fully duplicated rows and show some examples
dup_orders = orders_df.groupBy(orders_df.columns).count().filter(F.col("count") > 1)
print("Duplicate FULL rows count:", dup_orders.count())
dup_orders.show(20, truncate=False)
```

4) Reusable Cleaning Helpers

Several supplemental functions were defined to prevent redundancy and preserve code consistency:

- Each column's null values are counted by `show_null_counts(df)`.
- In categorical columns, `get_mode(df, col)` determines the most common (mode) value.
- `Impute_with_median(df, numeric_cols)`: uses the median to impute missing numerical values.
- `Impute_with_mean(df, numeric_cols)`: uses mean to impute missing numeric values.


```

# Generic helper to print number of nulls in each column (list format)
def show_null_counts(df):
    # Count nulls for every column
    null_counts_row = df.select([
        F.count(F.when(F.col(c).isNull(), c)).alias(c)
        for c in df.columns
    ]).collect()[0] # Get first row with all null counts
    # Print the result for each column as a list
    for col in df.columns:
        print(f"{col}: {null_counts_row[col]}")

# Generic helper to get the most frequent (mode) value of a column
def get_mode(df, col):
    mode_row = df.groupBy(col).count().orderBy(F.desc("count")).first()
    if mode_row is None:
        return None
    return mode_row[col]

# Generic helper to fill missing values in numeric columns using the column median
def impute_with_median(df, numeric_cols):
    medians = {}
    for c in numeric_cols:
        median = df.approxQuantile(c, [0.5], 0.01)[0] # Efficient Spark median
        medians[c] = median
    return df.na.fill(medians)

# Generic helper to fill missing values in numeric columns using the column mean
def impute_with_mean(df, numeric_cols):
    means = df.select([F.mean(c).alias(c) for c in numeric_cols]).collect()[0].asDict()
    return df.na.fill(means)

```

5) Dataset-Specific Cleaning Decisions

- **Orders Data:** We looked closely at missing timestamps in columns such as `order_approved_at`, `order_delivered_carrier_date`, and `order_delivered_customer_date`. To accurately depict genuine events, such as cancelled or pending orders, we kept nulls rather than filling these gaps with false values. This is crucial for honest downstream analysis.
- **Geolocation Data:** Since many buyers and sellers have the same zipcode prefix and location information, duplicate profiles were purposefully kept. Genuine spatial clustering is reflected in these duplicates, which is helpful when aggregating data for regional analysis later on.
- **Reviews Data:** Rows lacking essential keys, such as `order_id` or `review_id`, were eliminated to guarantee join integrity. The mean value—the most prevalent rating, usually 5, was used to impute missing review scores. Placeholders like "no text" or "no title" were used to fill in text boxes that lacked comments or titles. For time-based sentiment and performance assessments, a subset of reviews with complete date data was also produced.
- **Products Data:** Mode imputation was used to fill in missing values in product catalog columns such as `product_photos_qty`, `product_name_length`, and `product_description_length`. Median values were used to fill in numerical physical properties, such as weight and size, because this metric best maintains realistic distributions and is resistant to outliers.
- **Items and Payments Data:** We used medians to impute missing pricing and freight values in the items dataset and the mean to fill in the missing payment values in the payments dataset in order to ensure the correctness of subsequent revenue and financial computations. This method keeps aggregated data like average payments or total sales from being skewed by null values.

```

# Drop rows missing primary join keys (order_id or review_id)
reviews_clean = reviews_df.filter(
    F.col("order_id").isNotNull() & F.col("review_id").isNotNull()
)

# Impute missing review_score using mode (most common valid rating)
mode_score = get_mode(reviews_clean, "review_score")
reviews_clean = reviews_clean.na.fill({"review_score": mode_score})

# Fill missing review comment messages with "no text."
reviews_clean = reviews_clean.na.fill({"review_comment_message": "no text"})

# Fill missing review comment titles with "no title."
reviews_clean = reviews_clean.na.fill({"review_comment_title": "no title"})

# Keep rows with missing date fields (review_creation_date, review_answer_timestamp), Only drop these rows for time-based analyses:
reviews_dated = reviews_clean.filter(
    F.col("review_creation_date").isNotNull() & F.col("review_answer_timestamp").isNotNull()
)

```

```

# Mode imputation for categorical-ish columns
for col in ["product_photos_qty", "product_name_lenght", "product_description_lenght"]:
    mode_val = get_mode(products_df, col)
    if mode_val is None:
        mode_val = 0
    products_df = products_df.na.fill({col: mode_val})

# Median imputation for numerical product dimensions
products_df = impute_with_median(products_df, [
    "product_weight_g", "product_length_cm", "product_height_cm", "product_width_cm"
])

# Excluding is best for grouping, but not mandatory for overall product statistics.
products_with_category = products_df.filter(F.col("product_category_name").isNotNull())

```

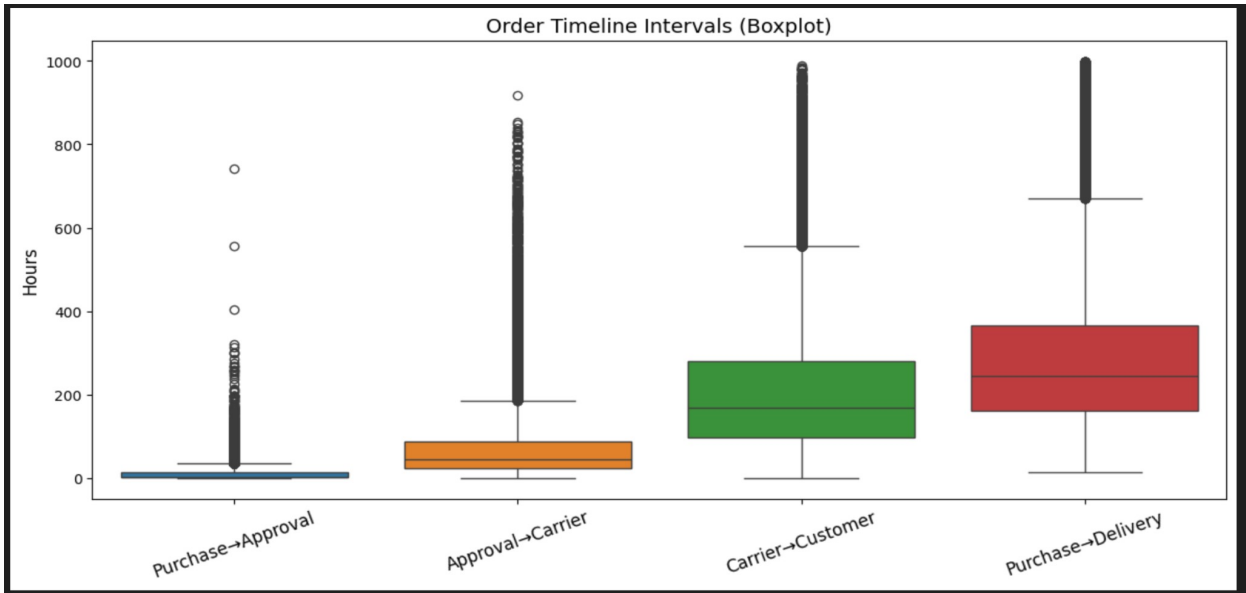
7. Development of the Pipeline

1. Order Timeline Analysis

Data Extraction Tool: To extract important timestamp fields, such as `order_purchase_timestamp`, `order_approved_at`, `order_delivered_carrier_date`, and `order_delivered_customer_date`, we utilized PySprak in the cleaned order dataset with headers recognized and schemas automatically inferred.

Data Processing Tool: Just these timestamps were used to generate a timeline DataFrame. After converting timestamps into epoch seconds using Spark's `unix_timestamp()` function, four interval KPIs were calculated in hours: purchase to approval, approval to carrier handoff, carrier to customer delivery, and total purchase to delivery time. To preserve data quality, rows containing null timestamps, negative intervals, or extreme outliers (>1000 hours) were eliminated. These

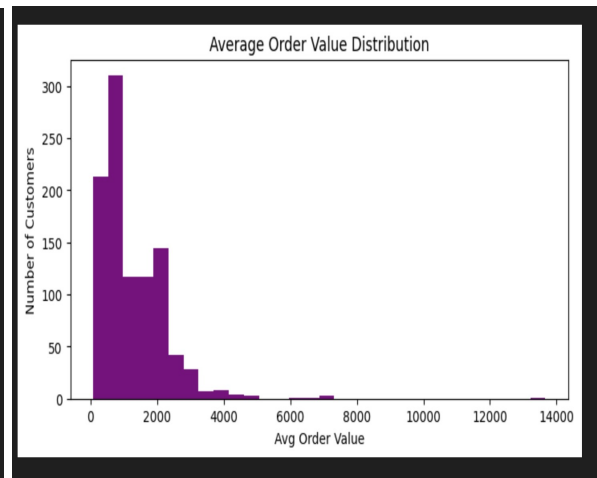
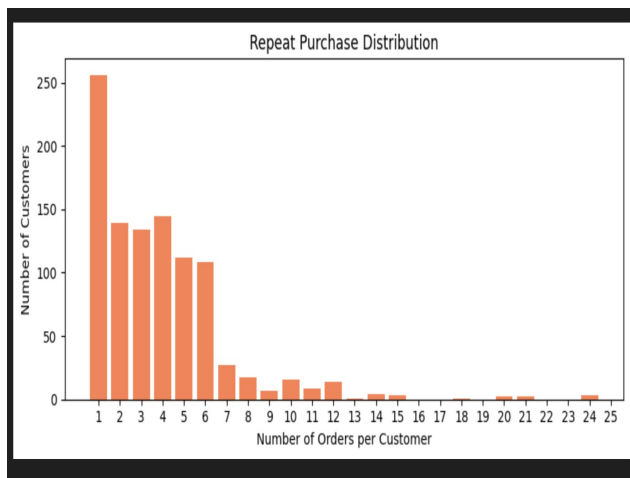
KPIs were displayed using histograms that were created directly from Spark DataFrames to highlight bottlenecks and delays.

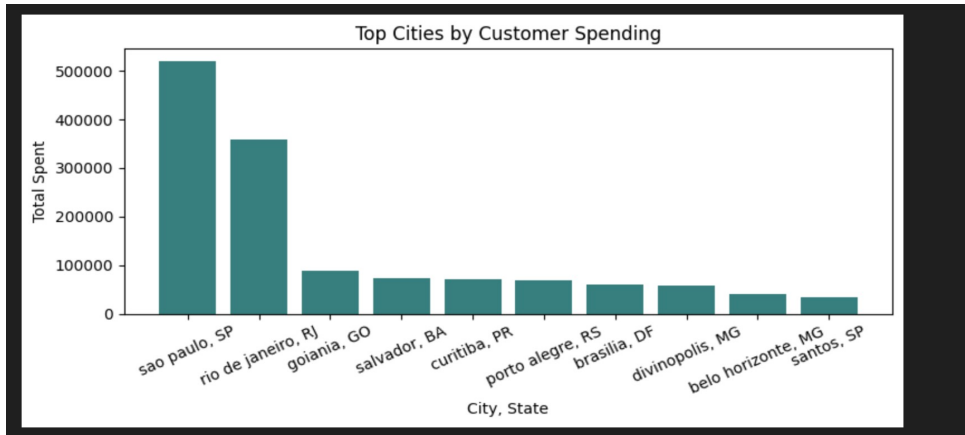


2. Customer Analysis

Data Extraction Tool: The combined enhanced orders DataFrame is used to facilitate scalability, querying, and aggregation. The DataFrame was registered as a Spark SQL temporary view.

Data Processing Tool: Spark SQL was used to calculate key customer KPIs, which were then grouped by customer_unique_id and enhanced with location context (city, state). Purchase count, amount spent (safely doubled), and average order value were among the metrics. Payment values that were invalid were excluded. To concentrate on high-value categories, results were sorted by total spend and restricted to the top 1000 clients. Histograms were used to display the distributions of purchase counts and total spending.

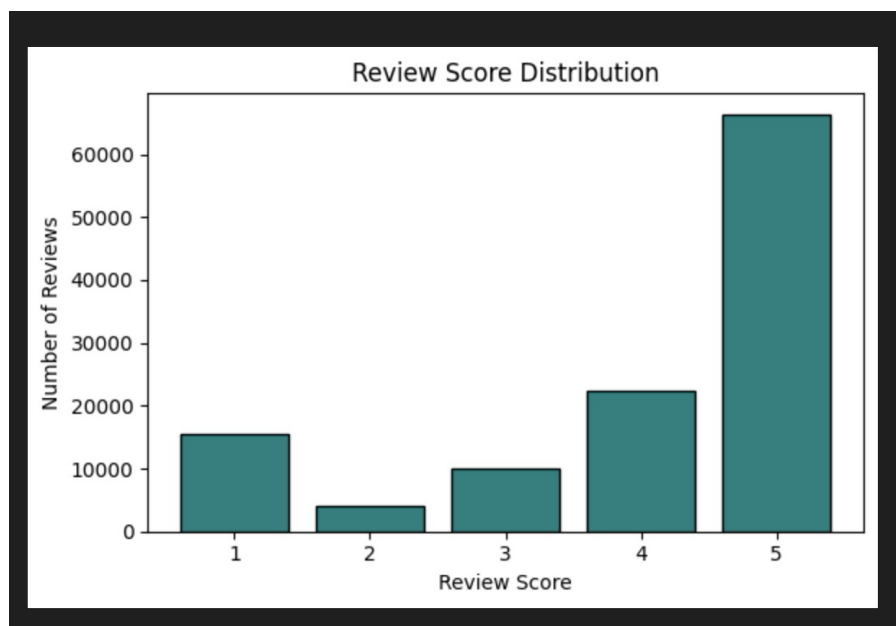




3. Review Quality and Sentiment

Data Extraction Tool: PySpark was used to load review-related fields. Missing review scores were imputed with mode values, and rows lacking primary keys (order_id or review_id) were eliminated. To allow for consistent analysis, placeholders were used in place of null review comment wording.

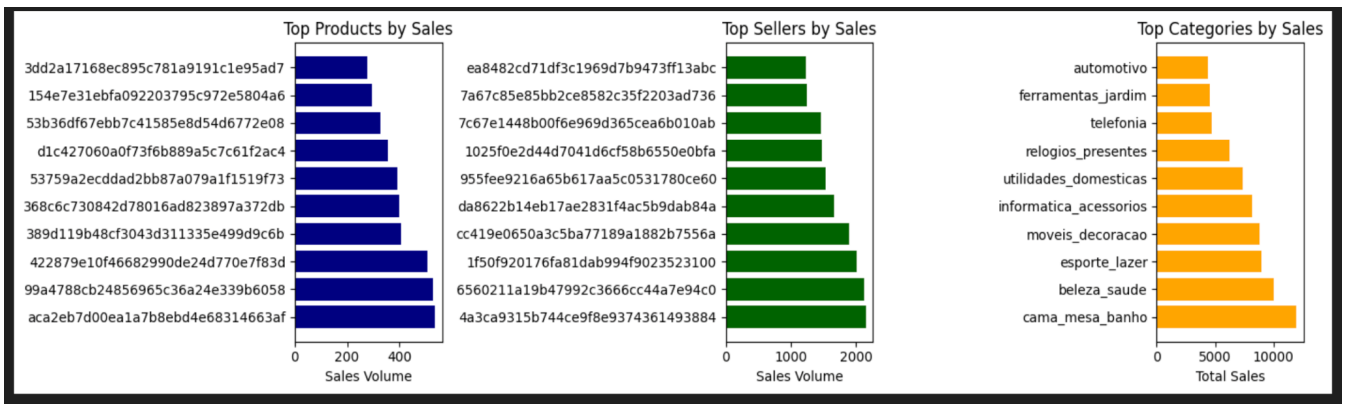
Data Processing Tool: Average review scores (~4.02) and review counts (~118,146) were determined by Spark aggregations. For proper analysis, the distribution of review scores (1–5) was calculated without nulls. Rules were used to assign sentiment labels (positive ≥ 4 , neutral =3, negative ≤ 2). Spark string processing methods were used to extract frequently occurring keywords from bad reviews. Score distributions were shown in bar charts, and frequent complaints were highlighted by keyword frequencies.



4. Product and Seller Performance

Data Extraction Tool: PySpark was used to load datasets of products, sellers, and enriched items from a merged CSV file and filter out records that lacked product or category IDs.

Data Processing Tool: Sales volume, average review rating, and return count (orders marked as returned) were calculated using PySpark aggregations by product, seller (with regional information), and product category. Horizontal bar charts were used to display the top 10 performances at each level, highlighting areas of possible quality or fulfillment risks as well as revenue drivers.



5. Geospatial Sales

Data Extraction Tool: The orders dataset was filtered to only include delivered orders. To facilitate spatial aggregation, geographic attributes (customer_city, customer_state, geo_lat12, geo_lng13) were included.

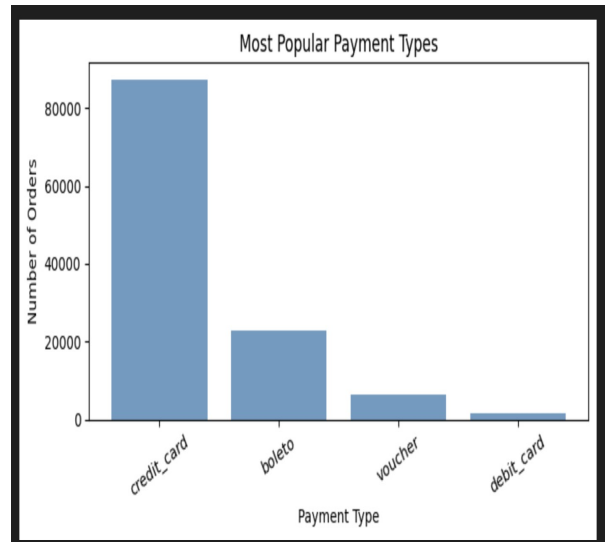
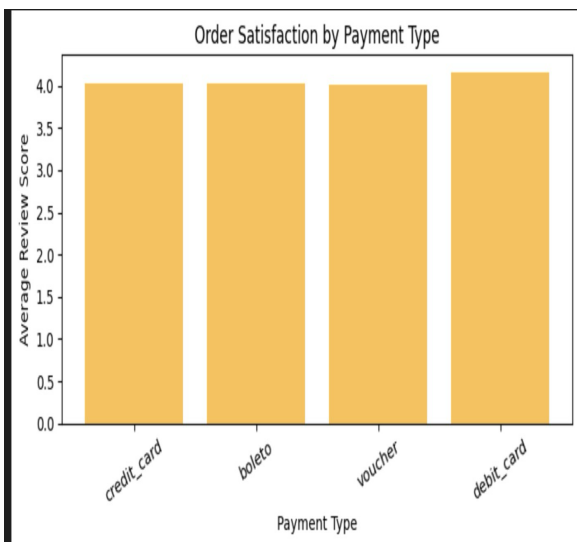
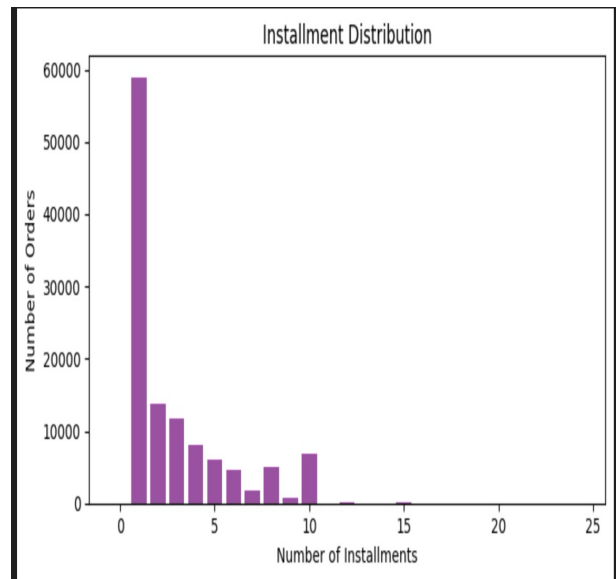
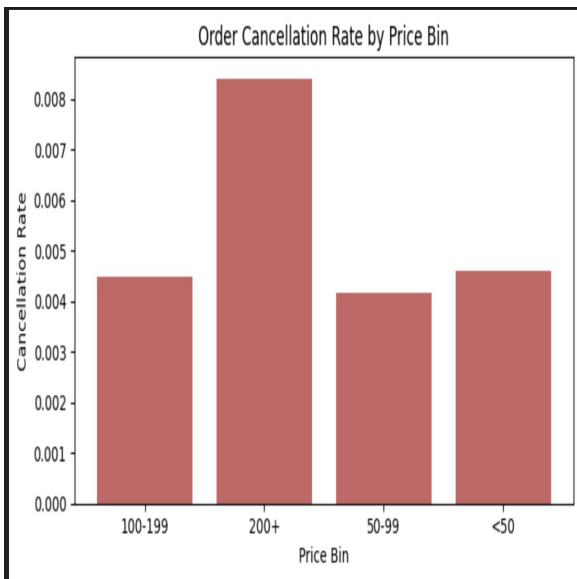
Data Processing Tool: Using Spark time functions, the difference in days between order_approved_at and order_delivered_customer_date was used to construct the delivery latency KPI (shipping_days). The number of sales, average delivery days, and average review scores were calculated using aggregations by city and state. The top 20 cities were evaluated based on sales volume in order to identify significant markets and logistical opportunities. Among the visualizations were location-based rankings and performance heatmaps.

Top 20 cities by sales volume:						
customer_city	customer_state	geo_lat12	geo_lng13	num_sales	avg_shipping_days	avg_review_score
brasil	DF	NULL	NULL	187	12.711443850267377	4.188172043010753
niteroi	RJ	-22.90581655	-43.10698886	154	14.236493506493506	3.835526315789474
rio de janeiro	RJ	-23.00927096	-43.42940928	149	20.892483221476514	3.5067567567567566
rio de janeiro	RJ	-23.01133469	-43.45025642	146	16.36835616438356	4.0069444444444445
niteroi	RJ	-22.91127038	-43.10515134	141	13.675602836879431	3.8652482269503547
lupatunga	MG	-19.47809554	-42.55679483	124	11.244112903225805	3.717741935483871
rio de janeiro	RJ	-22.07295652	-43.39706328	121	15.453223140495867	3.809917355371001
vila velha	ES	-20.33161264	-40.27958609	118	15.823559322033898	3.9909909090909091
campinas	SP	-22.85275771	-47.05510214	108	6.997037037037038	4.280373831775701
luberlandia	MG	-18.91595977	-48.27897417	103	9.989029126213593	4.326732673267327
vicosa	MG	-20.72193768	-42.8660116	103	11.230873786407768	3.796116504854369
jundiai	SP	-23.15171239	-46.97128781	103	8.150388349514564	4.223300970873787
lavras	MG	-21.24898019	-44.99817928	101	14.550990099090909	3.910891089108911
rio de janeiro	RJ	-23.00430869	-43.32450784	101	14.126831683168316	4.07
santos	SP	-23.98196407	-46.31003956	100	7.9004040404040403	4.13265306122449
litanhaem	SP	-24.14965101	-46.74809979	100	11.218800000000002	4.35
divinopolis	MG	-20.14661532	-44.89259168	99	10.051010101010101	4.1313131313131315
santos	SP	-23.96788955	-46.34412124	96	9.468333333333332	3.968421052631579
campinas	SP	-22.89587636	-47.07918503	95	8.650842105263159	3.5789473684210527
ubatuba	SP	-23.43162669	-45.0772287	95	11.488947368421053	3.925531914893617

6. Payment and Pricing Trends

Data Extraction Tool: To ensure that only valid entries are included, Spark's payment-related numeric fields (payment_value, payment_installments, price, and freight_value) were subjected to rule-based validation.

Data Processing Tool: computed order volume, average payment, review score, and installment count using Spark SQL to compile payment trends per method. By classifying orders according to the number of installments, installment patterns were examined and correlated with review scores. calculated variables such as delivery success, cancellations, average review, and freight, and created logical price bins. Visualizations revealed skewed installment usage, payment concentration by type, and higher cancellation risks in high-cost bins with sustained satisfaction levels after completion.



8. Challenges and Lessons Learned

Instead of analyzing a single, clean file, working with the Olist dataset seemed like building together a real e-commerce puzzle. The data came in eight different CSVs, and each one behaved differently. For example, orders had missing timestamps, reviews had optional content, payments and products had one-to-many relationships, and geolocation was enormous with repeating ZIP entries. As a result, merging required more than just "join and go"; we had to maintain consistent cleaning, handle missing information with caution, and ensure that the joins didn't inadvertently duplicate rows or inflate totals.

Managing geolocation was a major technical difficulty because directly joining it would result in more duplicates and more data. Another problem we encountered was that timeline computations fail when timestamps are lacking, so rather than imposing fictitious values, we filtered only the information required for time-based analysis. Lastly, we discovered that while Spark computes in a distributed manner, visualizing necessitates bringing smaller results to Python; hence, we had to aggregate first and only gather what we need.

Lessons learned: Good cleaning is a prerequisite for good insights. Geolocation requires preprocessing before joining; missing values should be treated according to meaning (not all nulls are "bad"), and when datasets have one-to-many relationships (orders–items–payments), we must always consider the proper “level” of analysis to prevent incorrect business conclusions.

9. References

i. Article: Applying Sentiment Analysis for Product Reviews and Predictions

V. Knap, “Applying Sentiment Analysis for Product Reviews and Predictions,” Jun. 5, 2019. [Online]. Available:

<https://vilemknap.cz/applying-sentiment-analysis-for-product-reviews-and-predictions/>.

ii. Blog: Business Intelligence Dashboards – Olist, The Brazilian E-Commerce Website

“Business Intelligence Dashboards: Olist, The Brazilian E-Commerce Website,” Data Visualization and Communication, Dec. 2, 2020. [Online]. Available:

<https://dataviz.etublogs.usj.edu.lb/2020/12/02/business-intelligence-dashboards-olist-the-brazilian-e-commerce-website/>.

iii. GitHub: Olist-E-commerce-Data-Analysis

T. Olist-E-commerce-Data-Analysis, “Olist-E-commerce-Data-Analysis,” GitHub, 2023. [Online]. Available:

<https://github.com/tolamoye/Olist-E-commerce-Data-Analysis>